

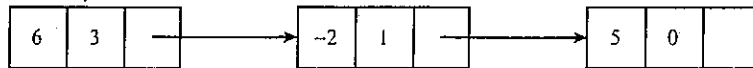
3. Escribir un programa que lea un texto de longitud indeterminada y que produzca como resultado la lista de todas las palabras diferentes contenidas en el texto, así como su frecuencia de aparición.
4. Escribir un programa que permita conocer el número de nodos de un árbol binario.
5. Un polinomio puede ser representado por una lista en la que cada nodo contiene un coeficiente, un exponente y un puntero o enlace hacia el siguiente elemento.

```

type
  Puntero = ^Nodo;
  Nodo = record
    Coeficiente : real;
    Exponente : integer;
    Enlace : Puntero;
  end;

```

Ejemplo: $6x^3 - 2x + 5$



Escribir un procedimiento que ejecute la suma y diferencia de dos polinomios almacenados de esta forma.

6. Se dispone de un árbol binario de elementos tipo *integer*. Escribir funciones que calculen:
 - a) La suma de sus elementos.
 - b) La suma de sus elementos que son múltiplos de 3.
7. Realizar una función *Idénticos* que permita decir si dos árboles binarios son iguales.
8. Utilizar una lista para controlar una lista de pasajeros de una línea aérea. El programa principal debe ser controlado por menú y permitir al usuario visualizar los datos de un pasajero determinado, visualizar la lista completa, crear una lista, insertar un nodo, borrar un nodo y sustituir los datos personales de un determinado pasajero.
9. Utilizar un árbol binario para ordenar un archivo de inventario de acuerdo al número de código del artículo y visualizar los datos.
10. Crear un archivo de datos en el que cada línea contenga lo siguiente:

Nombre	30 caracteres
Número de la Seguridad Social	10 caracteres
Dirección	24 caracteres

Escribir un programa que lea cada registro de datos en un árbol, de modo que cuando el árbol se recorra utilizando recorrido *en orden* los números de la Seguridad Social se almacenen en orden ascendente. Imprimir una cabecera "DATOS DE EMPLEADOS ORDENADOS DE ACUERDO AL NUMERO DE LA SEGURIDAD SOCIAL" y a continuación imprimir los datos del árbol con el formato Columnas.

1-10	Número de la Seguridad Social
20-50	Nombre
55-79	Dirección

CONSTRUCCION DE GRANDES PROGRAMAS (Las unidades y la compilación independiente)

CONTENIDO

- 18.1. Concepto de unidad
- 18.2. Estructura de una unidad
- 18.3. Creación de unidades
- 18.4. Utilización de unidad estándar
- 18.5. Situación de las unidades en sus discos:
¿Dónde busca Turbo Pascal las unidades?
- 18.6. Identificadores idénticos en diferentes unidades
- 18.7. Síntesis de unidades
- 18.8. Otros métodos de estructurar programas: inclusión, recubrimiento y encadenamiento

RESUMEN

PROBLEMAS

PROBLEMAS RESUELTOS

Una de las características más interesantes de Turbo Pascal 4/5 es la posibilidad de descomponer un programa grande en productos más pequeños que se pueden compilar independientemente. Estos módulos se denominan *unidades* y eliminan el inconveniente de las versiones anteriores a 4.0, de la limitación de la memoria ocupada por programas ejecutables a 64K. La versión 4.0 puede utilizar toda la memoria, hasta 640K. Los lectores que hayan utilizado Modula-2 encontrarán familiar el concepto unidad, ya que *unidad* (en Turbo Pascal 4.0 y 5.0/5.5) y *módulo* (en Modula-21) representan el concepto de compilación independiente y la posibilidad de construir grandes programas enlazando unidades compiladas independientemente para construir un programa ejecutable. Como resultado de ello, cuando se realiza un cambio en el código fuente, sólo se tiene que compilar el segmento al que afecta la modificación.

